# DATABASE SYSTEM
# MANAGE LEARNING GOALS OF STUDENTS
# FPT UNIVERSITY

ASSIGNMENT

**GROUP: 04**

STUDENT NAME: VU HONG QUAN | HE19xxxx
STUDENT NAME: NGUYEN VIET HOANG | HE19xxxx
STUDENT NAME: PHAN CONG MINH THANH | HE19xxxx

INSTRUCTOR: Dr. PHAN NGOC HOANG, Ph.D

November 07, 2025

# Contents


**1 INTRODUCE THE PROBLEM** **3**
1.1 DESCRIBE THE PROBLEM . . . 3
1.2 MANAGEMENT OBJECTIVES . . . 3

**2 ENTITY – RELATIONSHIP – ERD** **5**
2.1 DEFINITION ENTITY – ATTRIBUTE . . . 5
2.2 SET-UP ENTITY – RELATIONSHIP . . . 6

**3 DATA DICTIONARY** **8**
3.1 DATABASE AND TABLE . . . 8
3.1.1 CREATE DATABASE . . . 8
3.1.2 CREATE TABLE GoalCategoryLookup . . . 8
3.1.3 CREATE TABLE GoalStatusLookup . . . 9
3.1.4 CREATE TABLE SemesterLookup . . . 10
3.1.5 CREATE TABLE Instructor . . . 10
3.1.6 CREATE TABLE Student . . . 12
3.1.7 CREATE TABLE Course . . . 13
3.1.8 CREATE TABLE Enrollment . . . 14
3.1.9 CREATE TABLE LearningGoal . . . 15
3.1.10 CREATE TABLE ProgressTracking . . . 16
3.1.11 CREATE TABLE Milestone . . . 17
3.1.12 CREATE TABLE AdvisorAssignment . . . 18
3.1.13 CREATE TABLE AuditLog . . . 19

**4 ENTITY RELATIONSHIP DIAGRAM (ERD)** **20**
4.1 STUDENT . . . 20
4.2 INSTRUCTOR Entity . . . 21
4.3 COURSE Entity . . . 22
4.4 ENROLLMENT Entity . . . 23
4.5 LEARNINGGOAL Entity . . . 24
4.6 PROGRESSTRACKING Entity . . . 25
4.7 MILESTONE Entity . . . 26
4.8 ADVISORASSIGNMENT Entity . . . 27
4.9 LOOKUP TABLES . . . 28
4.9.1 GOALCATEGORYLOOKUP . . . 28
4.9.2 GOALSTATUSLOOKUP . . . 29
4.9.3 SEMESTERLOOKUP . . . 30
4.10 NORMAL FORM . . . 30
4.11 FULL DIAGRAM . . . 31

**5 SQL COMMAND** **32**
5.1 Query Using ORDER BY . . . . . 32
5.2 Query Using INNER JOIN . . . . . 33
5.3 Query Using AGGREGATE FUNCTIONS . . . . . 33
5.4 Query Using GROUP BY and HAVING Clauses . . . . . 34
5.5 Query Using a Sub-query as a Relation . . . . . 35
5.6 Query Using Partial Matching in WHERE Clause . . . . . 35
5.7 Query Using a Self-Join . . . . . 36
5.8 Stored Procedure . . . . . 38
5.9 Trigger (Audit Logging) . . . . . 39

# Chapter 1

# INTRODUCE THE PROBLEM

## 1.1 DESCRIBE THE PROBLEM

Nowadays, Students at FPT University set academic and personal goals to track their performance and skill development. However, there is no centralized database system to manage these objectives, track their progress, or link them to academic advisors. After analyzing the provided database schema (`FPTU_LearningGoalManagement`), the business rules are as follows:

- A student can define multiple learning goals. Each goal belongs to exactly one student. (1-N relationship from `Student` to `LearningGoal`).
- Each student can be assigned an `Instructor` as an academic advisor. This relationship is tracked over time, and a student can have only one active advisor at a time.
- The system records information about students, including: `StudentID` (e.g., 'SE160001'), `FirstName`, `LastName`, `Email`, `Major`, and `GPA`. The `GPA` must be between 0.00 and 4.00.
- A learning goal has a `GoalTitle`, `TargetValue` (e.g., 'GPA: 3.8'), `StartDate`, and `Deadline`. The `Deadline` must be later than the `StartDate`.
- Each goal is categorized (e.g., 'Academic', 'Skill Development') and has a status ('Active', 'Completed', 'Failed', 'Cancelled'), managed by lookup tables.
- Progress for a goal (`ProgressTracking`) can be updated by the student who owns the goal or by their currently assigned active advisor. Data consistency and security are ensured by triggers.
- A goal can have multiple `Milestone` (sub-tasks). The `DueDate` of a milestone cannot be after the `Deadline` of its parent goal.
- The system also manages student `Enrollment` in courses, recording the `Instructor`, `MidtermScore`, and `FinalScore` for each course in a specific `Semester`.

## 1.2 MANAGEMENT OBJECTIVES

- Manage student information and academic enrollments.
- Manage student-defined learning goals, milestones, and progress history.

- Manage the assignment of instructors as academic advisors.
- Manage course information, scores, and academic semesters.
- Enforce data consistency and security automatically through triggers.

**Important output:**

- A summary report of all student goals with their latest progress and time status.
- A calculated academic transcript showing average scores and grades.
- A report of all currently active student-advisor pairings.
- A detailed report of milestone status (Pending, Completed, Overdue) for each goal.

# Chapter 2

# ENTITY – RELATIONSHIP – ERD

## 2.1 DEFINITION ENTITY – ATTRIBUTE

Based on the problem description and management objectives from Chapter I, we can present several entities and attributes of the entity as follows

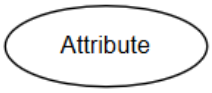
- **GoalCategoryLookup**: CategoryID (PK), CategoryName, Description, IsActive
- **GoalStatusLookup**: StatusID (PK), StatusName, Description, IsActive
- **SemesterLookup**: SemesterID (PK), SemesterCode, StartDate, EndDate, IsActive
- **Instructor**: InstructorID (PK), FirstName, LastName, Email, Department, Phone, HireDate, IsActive
- **Student**: StudentID (PK), FirstName, LastName, Email, Major, GPA, EnrollmentDate, Phone, IsActive
- **Course**: CourseID (PK), CourseName, Credits, Description, IsActive
- **Enrollment**: EnrollmentID (PK), StudentID (FK), CourseID (FK), SemesterID (FK), InstructorID (FK), MidtermScore, FinalScore, EnrollmentDate
- **LearningGoal**: GoalID (PK), StudentID (FK), CategoryID (FK), GoalTitle, GoalDescription, TargetValue, StartDate, Deadline, StatusID (FK), CreatedDate, LastModifiedDate
- **ProgressTracking**: TrackingID (PK), GoalID (FK), ProgressPercent, Notes, TrackingDate, UpdatedBy
- **Milestone**: MilestoneID (PK), GoalID (FK), MilestoneName, Description, DueDate, CompletedDate, CreatedDate
- **AdvisorAssignment**: AssignmentID (PK), StudentID (FK), InstructorID (FK), AssignedDate, UnassignedDate

## 2.2 SET-UP ENTITY – RELATIONSHIP

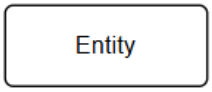
### Some symbols used in the model

- Key / identifier attribute

- Attribute description / description

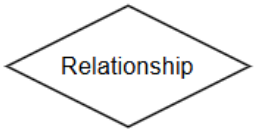



- Entity




- Weak entity

- Relationship




- Connectivity (force) = 1



- Connectivity = N

*Entity Relationship Diagram Symbols*

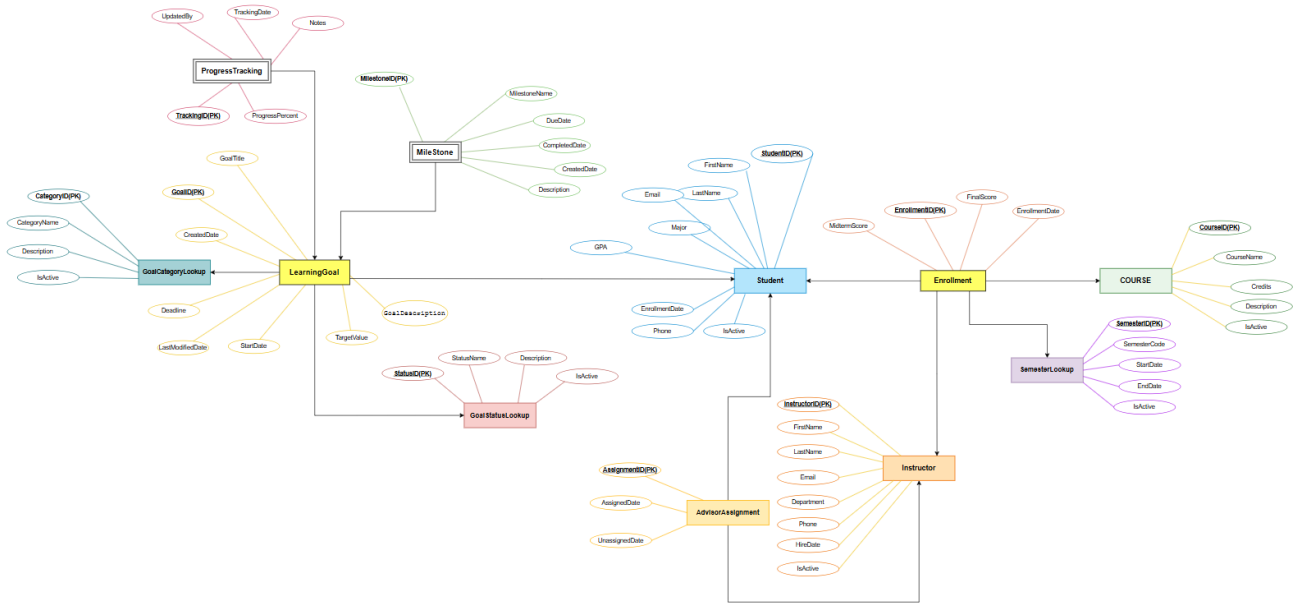



Figure 2.1: Entity Relationship Diagram

# Chapter 3

# DATA DICTIONARY

## 3.1 DATABASE AND TABLE

### 3.1.1 CREATE DATABASE

Listing 3.1: Create Database (`FPTU_LearningGoalManagement`)

```sql
-- Create database FPTU_LearningGoalManagement
CREATE DATABASE FPTU_LearningGoalManagement;
```

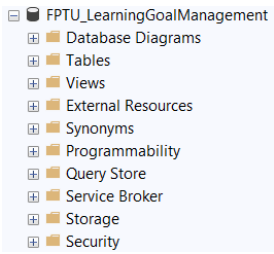



Figure 3.1: Entity Relationship Diagram

### 3.1.2 CREATE TABLE GoalCategoryLookup

**Column Definition:GoalCategoryLookup**

| Column Name | Data Type | Default | Check | Key/ Index/ Constraint |
|---|---|---|---|---|
| CategoryID | INT | | | PRIMARY KEY, IDENTITY(1,1) |
| CategoryName | NVARCHAR(50) | | | NOT NULL, UNIQUE |
| Description | NVARCHAR(200) | | | |
| IsActive | BIT | 1 | | |

Table 3.1: Table Schema for GoalCategoryLookup

Listing 3.2: Create Table GoalCategoryLookup

```sql
-- Create table guard
    CREATE TABLE GoalCategoryLookup (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(50) NOT NULL UNIQUE,
    Description NVARCHAR(200),
    IsActive BIT DEFAULT 1
);
```

**Example Data**

| CategoryID | CategoryName | Description | IsActive |
|---|---|---|---|
| 1 | Academic | Goals related to study, GPA, and grades | 1 |
| 2 | Skill Development | Professional and technical skill development | 1 |
| 3 | Career | Career goals, internships, and employment | 1 |
| 4 | Personal | Personal goals, health, and self-improvement | 1 |
| 5 | Project | Project completion and assignment goals | 1 |
| 6 | Certificate | Professional certification goals | 1 |

Table 3.2: Data for GoalCategoryLookup

### 3.1.3 CREATE TABLE GoalStatusLookup

**Column Definition: GoalStatusLookup**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| StatusID | INT | | | PK, IDENTITY(1,1) |
| StatusName | NVARCHAR(30) | | IN values | NOT NULL, UNIQUE |
| Description | NVARCHAR(200) | | | |
| IsActive | BIT | 1 | | |

Table 3.3: Table Schema for GoalStatusLookup

Listing 3.3: Create Table GoalStatusLookup

```sql
CREATE TABLE GoalStatusLookup (
    StatusID INT PRIMARY KEY IDENTITY(1,1),
    StatusName NVARCHAR(30) NOT NULL UNIQUE
        CHECK(StatusName IN ('Active', 'Completed',
                             'Failed', 'Cancelled')),
    Description NVARCHAR(200),
    IsActive BIT DEFAULT 1
);
```

**Example Data**

| StatusID | StatusName | Description | IsActive |
|---|---|---|---|
| 1 | Active | Goal is currently in progress | 1 |
| 2 | Completed | Goal has been successfully completed | 1 |
| 3 | Failed | Goal has failed | 1 |
| 4 | Cancelled | Goal has been cancelled | 1 |

Table 3.4: Sample Data for GoalStatusLookup

### 3.1.4 CREATE TABLE SemesterLookup

**Column Definition: SemesterLookup**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| SemesterID | INT | | | PK, IDENTITY(1,1) |
| SemesterCode | VARCHAR(20) | | | NOT NULL, UNIQUE |
| StartDate | DATE | | | NOT NULL |
| EndDate | DATE | | EndDate > Start | NOT NULL |
| IsActive | BIT | 1 | | |

Table 3.5: Table Schema for SemesterLookup

Listing 3.4: Create Table SemesterLookup

```sql
CREATE TABLE SemesterLookup (
    SemesterID INT PRIMARY KEY IDENTITY(1,1),
    SemesterCode VARCHAR(20) NOT NULL UNIQUE,
    StartDate DATE NOT NULL,
    EndDate DATE NOT NULL,
    IsActive BIT DEFAULT 1,
    CHECK(EndDate > StartDate)
);
```

**Example Data**

| SemesterID | SemesterCode | StartDate | EndDate | IsActive |
|---|---|---|---|---|
| 1 | Fall2024 | 2024-09-01 | 2024-12-31 | 1 |
| 2 | Spring2025 | 2025-01-01 | 2025-05-31 | 1 |
| 3 | Summer2025 | 2025-06-01 | 2025-08-31 | 1 |
| 4 | Fall2025 | 2025-09-01 | 2025-12-31 | 1 |

Table 3.6: Sample Data for SemesterLookup

### 3.1.5 CREATE TABLE Instructor

**Column Definition: Instructor**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| InstructorID | VARCHAR(10) | | LIKE pattern | PK |
| FirstName | NVARCHAR(50) | | | NOT NULL |
| LastName | NVARCHAR(50) | | | NOT NULL |
| Email | NVARCHAR(100) | | Email valid | UNIQUE, NOT NULL |
| Department | NVARCHAR(100) | | | NOT NULL |
| Phone | VARCHAR(15) | | | |
| HireDate | DATE | GETDATE() | | |
| IsActive | BIT | 1 | | |

Table 3.7: Table Schema for Instructor

Listing 3.5: Create Table Instructor

```sql
CREATE TABLE Instructor (
    InstructorID VARCHAR(10) PRIMARY KEY
        CHECK(InstructorID LIKE 'INS[0-9][0-9][0-9][0-9][0-9]'),
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE
        CHECK(Email LIKE '%_@__%.__%'),
    Department NVARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    HireDate DATE DEFAULT GETDATE(),
    IsActive BIT DEFAULT 1
);
```

### Example Data

| InstructorID | FirstName | LastName | Email | Department |
|---|---|---|---|---|
| INS00001 | John | Smith | john.smith@fptu.edu.vn | Software Engineering |
| INS00002 | Mary | Johnson | mary.johnson@fptu.edu.vn | Information Systems |
| INS00003 | David | Williams | david.williams@fptu.edu.vn | Artificial Intelligence |
| INS00004 | Sarah | Brown | sarah.brown@fptu.edu.vn | Software Engineering |
| INS00005 | Robert | Davis | robert.davis@fptu.edu.vn | Data Science |

Table 3.8: Sample Data for Instructor

### 3.1.6 CREATE TABLE Student

**Column Definition: Student**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| StudentID | VARCHAR(10) | | LIKE pattern | PK |
| FirstName | NVARCHAR(50) | | | NOT NULL |
| LastName | NVARCHAR(50) | | | NOT NULL |
| Email | NVARCHAR(100) | | Email valid | UNIQUE, NOT NULL |
| Major | NVARCHAR(100) | | | NOT NULL |
| GPA | DECIMAL(3,2) | 0.00 | 0.00–4.00 | |
| EnrollmentDate | DATE | GETDATE() | | |
| Phone | VARCHAR(15) | | | |
| IsActive | BIT | 1 | | |

Table 3.9: Table Schema for Student

Listing 3.6: Create Table Student

```sql
CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY
        CHECK(
            (StudentID LIKE 'SE[0-9][0-9][0-9][0-9][0-9][0-9]')
            OR (StudentID LIKE 'HE[0-9][0-9][0-9][0-9][0-9][0-9]')
            OR (StudentID LIKE 'SS[0-9][0-9][0-9][0-9][0-9][0-9]')
        ),
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE
        CHECK(Email LIKE '%_@__%.__%'),
    Major NVARCHAR(100) NOT NULL,
    GPA DECIMAL(3,2) DEFAULT 0.00
        CHECK(GPA >= 0.00 AND GPA <= 4.00),
    EnrollmentDate DATE DEFAULT GETDATE(),
    Phone VARCHAR(15),
    IsActive BIT DEFAULT 1
);
```

**Example Data**

| StudentID | FirstName | LastName | Email | Major | GPA |
|---|---|---|---|---|---|
| SE160001 | Nguyen | Van A | anvse160001@fpt.edu.vn | Software Engineering | 3.50 |
| SE160002 | Tran | Thi B | bttse160002@fpt.edu.vn | Software Engineering | 3.75 |
| HE160001 | Pham | Thi D | dtphe160001@fpt.edu.vn | Information Systems | 3.60 |
| HE160002 | Hoang | Minh E | emhe160002@fpt.edu.vn | Information Systems | 3.40 |
| SS160001 | Vu | Thanh F | ftvss160001@fpt.edu.vn | Software Engineering | 3.80 |

Table 3.10: Sample Data for Student

### 3.1.7 CREATE TABLE Course

**Column Definition: Course**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| CourseID | VARCHAR(10) | | LIKE pattern | PK |
| CourseName | NVARCHAR(200) | | | NOT NULL |
| Credits | INT | | 1–6 | NOT NULL |
| Description | NVARCHAR(500) | | | |
| IsActive | BIT | 1 | | |

Table 3.11: Table Schema for Course

Listing 3.7: Create Table Course

```sql
CREATE TABLE Course (
    CourseID VARCHAR(10) PRIMARY KEY
        CHECK(CourseID LIKE '[A-Z][A-Z][A-Z][0-9][0-9][0-9]'),
    CourseName NVARCHAR(200) NOT NULL,
    Credits INT NOT NULL CHECK(Credits > 0 AND Credits <= 6),
    Description NVARCHAR(500),
    IsActive BIT DEFAULT 1
);
```

**Example Data**

| CourseID | CourseName | Credits | Description |
|---|---|---|---|
| DBI202 | Introduction to Databases | 3 | Database design, SQL, normalization |
| PRJ301 | Java Web Application | 3 | Web application development |
| SWE201 | Software Engineering | 3 | SDLC, Agile, processes |
| MAE101 | Math for Engineers | 4 | Advanced math, linear algebra |
| AID101 | AI Basics | 3 | Introduction to AI and ML |

Table 3.12: Sample Data for Course

### 3.1.8 CREATE TABLE Enrollment

**Column Definition: Enrollment**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| EnrollmentID | INT | | | PK, IDENTITY(1,1) |
| StudentID | VARCHAR(10) | | | FK, NOT NULL |
| CourseID | VARCHAR(10) | | | FK, NOT NULL |
| SemesterID | INT | | | FK, NOT NULL |
| InstructorID | VARCHAR(10) | NULL | | FK |
| MidtermScore | DECIMAL(5,2) | | 0–10 | |
| FinalScore | DECIMAL(5,2) | | 0–10 | |
| EnrollmentDate | DATE | GETDATE() | | |

Table 3.13: Table Schema for Enrollment

Listing 3.8: Create Table Enrollment

```sql
CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY IDENTITY(1,1),
    StudentID VARCHAR(10) NOT NULL,
    CourseID VARCHAR(10) NOT NULL,
    SemesterID INT NOT NULL,
    InstructorID VARCHAR(10) NULL,
    MidtermScore DECIMAL(5,2)
        CHECK(MidtermScore >= 0 AND MidtermScore <= 10),
    FinalScore DECIMAL(5,2)
        CHECK(FinalScore >= 0 AND FinalScore <= 10),
    EnrollmentDate DATE DEFAULT GETDATE(),
    UNIQUE(StudentID, CourseID, SemesterID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
        ON DELETE NO ACTION ON UPDATE CASCADE,
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
        ON DELETE NO ACTION ON UPDATE CASCADE,
    FOREIGN KEY (SemesterID) REFERENCES SemesterLookup(SemesterID)
        ON DELETE NO ACTION,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
        ON DELETE SET NULL
);
```

**Example Data**

| EnrollID | StudentID | CourseID | SemID | InstructorID | Midterm | Final |
|---|---|---|---|---|---|---|
| 1 | SE160001 | DBI202 | 1 | INS00001 | 8.5 | 9.0 |
| 2 | SE160001 | PRJ301 | 1 | INS00002 | 7.5 | 8.0 |
| 3 | SE160002 | DBI202 | 1 | INS00001 | 9.0 | 9.5 |
| 9 | SE160001 | WEB301 | 2 | INS00002 | NULL | NULL |
| 10 | SE160002 | MLE401 | 2 | INS00005 | 8.8 | 9.2 |

Table 3.14: Sample Data for Enrollment

### 3.1.9 CREATE TABLE LearningGoal

**Column Definition: LearningGoal**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| GoalID | INT | | | PK, IDENTITY(1,1) |
| StudentID | VARCHAR(10) | | | FK, NOT NULL |
| CategoryID | INT | | | FK, NOT NULL |
| GoalTitle | NVARCHAR(200) | | | NOT NULL |
| GoalDescription | NVARCHAR(500) | | | |
| TargetValue | NVARCHAR(100) | | | NOT NULL |
| StartDate | DATE | GETDATE() | | NOT NULL |
| Deadline | DATE | | Deadline > Start | NOT NULL |
| StatusID | INT | 1 | | FK, NOT NULL |
| CreatedDate | DATETIME | GETDATE() | | |
| LastModifiedDate | DATETIME | GETDATE() | | |

Table 3.15: Table Schema for LearningGoal

Listing 3.9: Create Table LearningGoal

```sql
CREATE TABLE LearningGoal (
    GoalID INT PRIMARY KEY IDENTITY(1,1),
    StudentID VARCHAR(10) NOT NULL,
    CategoryID INT NOT NULL,
    GoalTitle NVARCHAR(200) NOT NULL,
    GoalDescription NVARCHAR(500),
    TargetValue NVARCHAR(100) NOT NULL,
    StartDate DATE NOT NULL DEFAULT GETDATE(),
    Deadline DATE NOT NULL,
    StatusID INT NOT NULL DEFAULT 1,
    CreatedDate DATETIME DEFAULT GETDATE(),
    LastModifiedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
        ON DELETE NO ACTION ON UPDATE CASCADE,
    FOREIGN KEY (CategoryID)
        REFERENCES GoalCategoryLookup(CategoryID),
    FOREIGN KEY (StatusID)
        REFERENCES GoalStatusLookup(StatusID),
    CHECK(Deadline > StartDate)
);
```

**Example Data**

| GoalID | StudentID | Cat | GoalTitle | TargetValue | Deadline |
|---|---|---|---|---|---|
| 1 | SE160001 | 1 | Achieve GPA 3.8 | GPA: 3.8 | 2024-12-31 |
| 2 | SE160001 | 2 | AWS Certification | Certificate | 2025-03-31 |
| 3 | SE160001 | 3 | FPT Internship | Job Offer | 2025-02-28 |
| 4 | SE160002 | 1 | Maintain GPA 3.7 | GPA: 3.7+ | 2024-12-31 |
| 5 | SE160002 | 5 | Capstone Project | Grade: A | 2025-05-31 |

Table 3.16: Sample Data for LearningGoal

### 3.1.10 CREATE TABLE ProgressTracking

**Column Definition: ProgressTracking**

*Design Decision (Security):* UpdatedBy is VARCHAR (not FK) to allow both StudentID and InstructorID.

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| TrackingID | INT | | | PK, IDENTITY(1,1) |
| GoalID | INT | | | FK, NOT NULL |
| ProgressPercent | DECIMAL(5,2) | | 0–100 | NOT NULL |
| Notes | NVARCHAR(500) | | | |
| TrackingDate | DATETIME | GETDATE() | | NOT NULL |
| UpdatedBy | VARCHAR(10) | | | NOT NULL |

Table 3.17: Table Schema for ProgressTracking

Listing 3.10: Create Table ProgressTracking

```sql
CREATE TABLE ProgressTracking (
    TrackingID INT PRIMARY KEY IDENTITY(1,1),
    GoalID INT NOT NULL,
    ProgressPercent DECIMAL(5,2) NOT NULL
        CHECK(ProgressPercent >= 0 AND ProgressPercent <= 100),
    Notes NVARCHAR(500),
    TrackingDate DATETIME NOT NULL DEFAULT GETDATE(),
    UpdatedBy VARCHAR(10) NOT NULL,
    FOREIGN KEY (GoalID) REFERENCES LearningGoal(GoalID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

**Example Data**

| TrackingID | GoalID | Progress% | Notes | UpdatedBy |
|---|---|---|---|---|
| 1 | 1 | 25.00 | Completed 2 of 8 courses, average 3.5 | SE160001 |
| 2 | 1 | 50.00 | Midterm results good, current GPA 3.6 | SE160001 |
| 3 | 1 | 75.00 | Continued progress, 5 of 8 courses completed | INS00001 |
| 4 | 1 | 100.00 | Completed. Final GPA 3.82 | SE160001 |
| 5 | 2 | 20.00 | Completed module 1 on Udemy | SE160001 |

Table 3.18: Sample Data for ProgressTracking

### 3.1.11 CREATE TABLE Milestone

**Column Definition: Milestone**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| MilestoneID | INT | | | PK, IDENTITY(1,1) |
| GoalID | INT | | | FK, NOT NULL |
| MilestoneName | NVARCHAR(200) | | | NOT NULL |
| Description | NVARCHAR(500) | | | |
| DueDate | DATE | | | NOT NULL |
| CompletedDate | DATE | | Comp $\geq$ Created | |
| CreatedDate | DATETIME | | | NOT NULL |

Table 3.19: Table Schema for Milestone

Listing 3.11: Create Table Milestone

```sql
CREATE TABLE Milestone (
    MilestoneID INT PRIMARY KEY IDENTITY(1,1),
    GoalID INT NOT NULL,
    MilestoneName NVARCHAR(200) NOT NULL,
    Description NVARCHAR(500),
    DueDate DATE NOT NULL,
    CompletedDate DATE,
    CreatedDate DATETIME NOT NULL,
    CHECK(CompletedDate IS NULL
          OR CompletedDate >= CreatedDate),
    FOREIGN KEY (GoalID) REFERENCES LearningGoal(GoalID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

**Example Data**

| MilestoneID | GoalID | MilestoneName | DueDate | CompletedDate | CreatedDate |
|---|---|---|---|---|---|
| 1 | 1 | Midterm Exams | 2024-10-31 | 2024-10-30 | 2024-09-01 |
| 2 | 1 | Final Exams | 2024-12-20 | 2024-12-25 | 2024-09-01 |
| 3 | 1 | Final GPA Check | 2024-12-31 | 2024-12-26 | 2024-09-01 |
| 4 | 2 | Complete AWS Course | 2024-12-31 | NULL | 2024-09-01 |
| 10 | 5 | Design Document | 2024-10-30 | 2024-10-28 | 2024-09-01 |

Table 3.20: Sample Data for Milestone

### 3.1.12 CREATE TABLE AdvisorAssignment

**Column Definition: AdvisorAssignment**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| AssignmentID | INT | | | PK, IDENTITY(1,1) |
| StudentID | VARCHAR(10) | | | FK, NOT NULL |
| InstructorID | VARCHAR(10) | | | FK, NOT NULL |
| AssignedDate | DATE | GETDATE() | | NOT NULL |
| UnassignedDate | DATE | | Unassign > Assign | |

Table 3.21: Table Schema for AdvisorAssignment

Listing 3.12: Create Table AdvisorAssignment

```sql
CREATE TABLE AdvisorAssignment (
    AssignmentID INT PRIMARY KEY IDENTITY(1,1),
    StudentID VARCHAR(10) NOT NULL,
    InstructorID VARCHAR(10) NOT NULL,
    AssignedDate DATE NOT NULL DEFAULT GETDATE(),
    UnassignedDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
        ON DELETE NO ACTION ON UPDATE CASCADE,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
        ON DELETE NO ACTION ON UPDATE CASCADE,
    CHECK(UnassignedDate IS NULL
          OR UnassignedDate > AssignedDate)
);
```

**Example Data**

| AssignmentID | StudentID | InstructorID | AssignedDate | UnassignedDate | Status |
|---|---|---|---|---|---|
| 1 | SE160001 | INS00001 | 2024-09-01 | NULL | Active |
| 2 | SE160002 | INS00001 | 2024-09-01 | NULL | Active |
| 3 | HE160001 | INS00002 | 2024-09-01 | 2025-01-31 | Inactive |
| 4 | HE160002 | INS00003 | 2024-09-01 | NULL | Active |
| 5 | SS160001 | INS00004 | 2024-09-01 | NULL | Active |

Table 3.22: Sample Data for AdvisorAssignment

### 3.1.13 CREATE TABLE AuditLog

**Column Definition: AuditLog**

| Column Name | Data Type | Default | Check | Key/Constraint |
|---|---|---|---|---|
| LogID | INT | | | PK, IDENTITY(1,1) |
| TableName | NVARCHAR(50) | | | NOT NULL |
| Operation | NVARCHAR(20) | | IN (I,U,D) | NOT NULL |
| RecordID | NVARCHAR(100) | | | NOT NULL |
| OldValue | NVARCHAR(MAX) | | | |
| NewValue | NVARCHAR(MAX) | | | |
| ModifiedBy | VARCHAR(10) | | | |
| ModifiedDate | DATETIME | GETDATE() | | |

Table 3.23: Table Schema for AuditLog

Listing 3.13: Create Table AuditLog

```sql
CREATE TABLE AuditLog (
    LogID INT PRIMARY KEY IDENTITY(1,1),
    TableName NVARCHAR(50) NOT NULL,
    Operation NVARCHAR(20) NOT NULL
        CHECK(Operation IN ('INSERT', 'UPDATE', 'DELETE')),
    RecordID NVARCHAR(100) NOT NULL,
    OldValue NVARCHAR(MAX),
    NewValue NVARCHAR(MAX),
    ModifiedBy VARCHAR(10),
    ModifiedDate DATETIME DEFAULT GETDATE()
);
```

**Example Data**

| LogID | TableName | Operation | RecordID | ModifiedBy |
|---|---|---|---|---|
| 1 | LearningGoal | INSERT | 1 | SE160001 |
| 2 | LearningGoal | UPDATE | 1 | SE160001 |
| 3 | ProgressTracking | INSERT | 4 | SE160001 |
| 4 | Enrollment | INSERT | 1 | SYSTEM |
| 5 | AdvisorAssignment | UPDATE | 4 | HE160001 |

Table 3.24: Sample Data for AuditLog

# Chapter 4

# ENTITY RELATIONSHIP DIAGRAM (ERD)

## 4.1 STUDENT

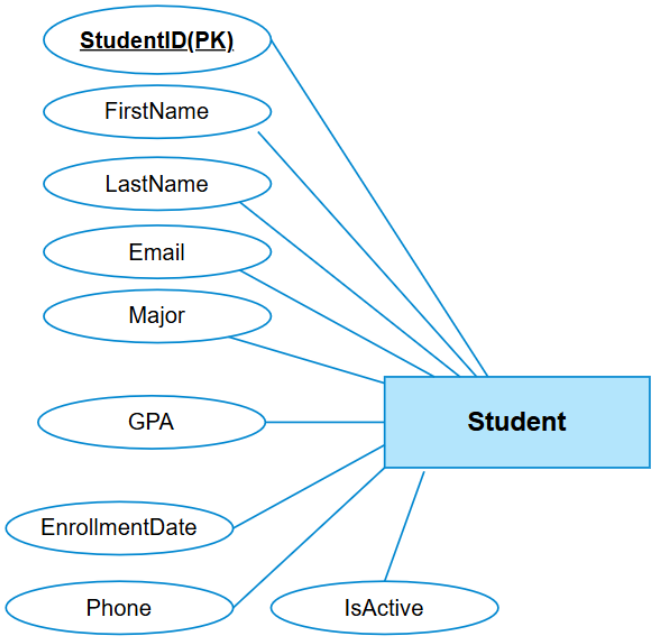
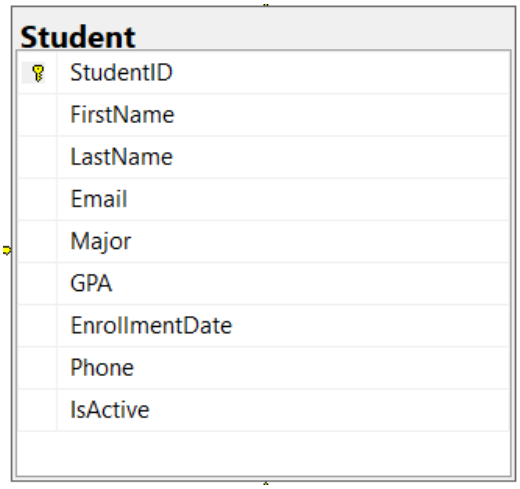



Figure 4.1: Student Entity

This is the Student entity, representing students enrolled in the university. The Student entity has 9 attributes: StudentID (PK - Primary Key), FirstName, LastName, Email, Major, GPA, EnrollmentDate, Phone, and IsActive.




## 4.2 INSTRUCTOR Entity

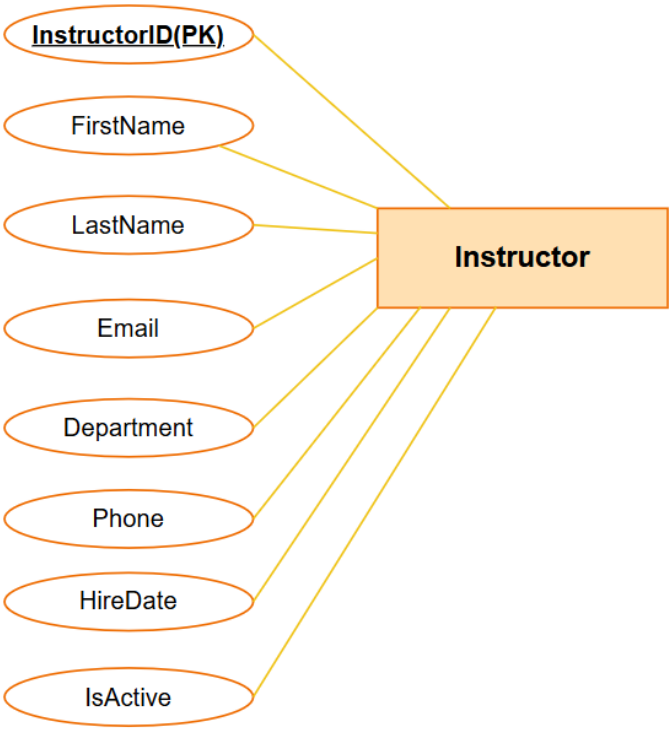
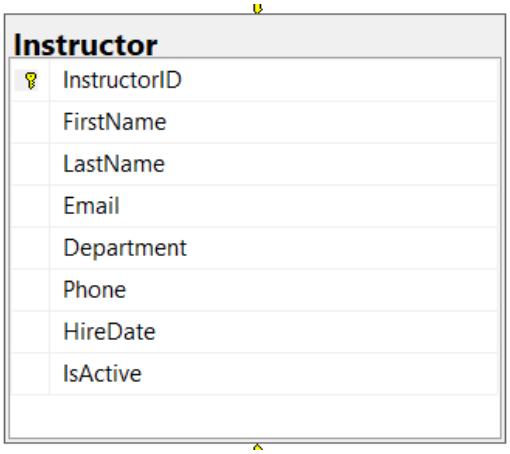



Figure 4.2:

This is the Instructor entity, the secondary root of the diagram—faculty members who teach courses and advise students. Instructor has 8 attributes. InstructorID uniquely identifies each instructor using the format INS[0-9]5 (e.g., INS00001, INS00042). Each instructor has Name information split into FirstName and LastName, enabling professional communication. Contact information comprises Email (UNIQUE, format "[firstname].[lastname]@fptu.edu.vn") and Phone. The Department attribute identifies the instructor's organizational unit (Software Engineering, Information Systems, Artificial Intelligence, Data Science). HireDate records employment start date, tracking tenure. IsActive indicates current employment status (1=Active, 0=Retired/On-leave). Only active instructors can teach courses and serve as student advisors.




## 4.3 COURSE Entity

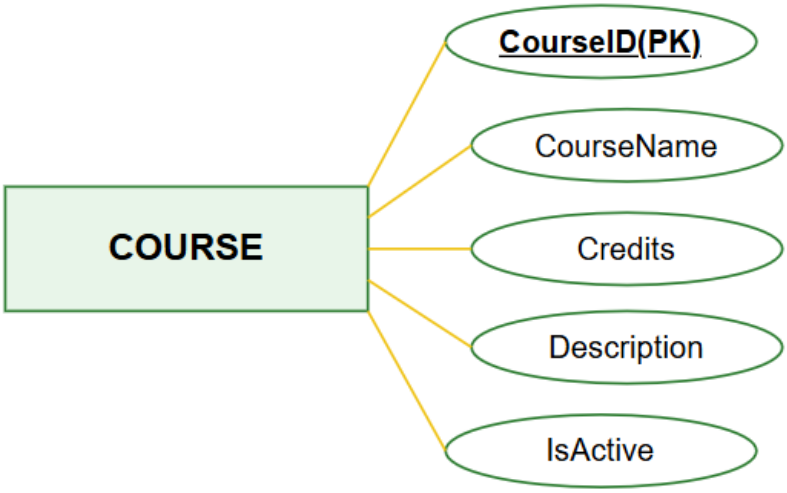
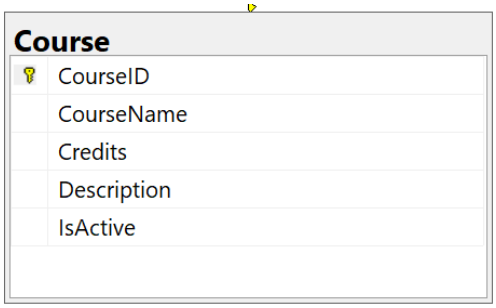



Figure 4.3:

This is the Course entity, a static reference entity representing the course catalog. Course has 5 attributes. CourseID is the primary key using the format [A-Z][A-Z][A-Z][0-9][0-9][0-9] (e.g., DBI202, PRJ301, SWE201). CourseName is the full name of the course (e.g., "Introduction to Databases"). Credits represents the academic credit hours, ranging from 1 to 6. Description provides detailed course objectives and content. IsActive indicates whether the course is currently offered (1) or discontinued (0). One course can have many enrollments across different semesters and instructors. This entity is independent and does not depend on other entities.




## 4.4 ENROLLMENT Entity

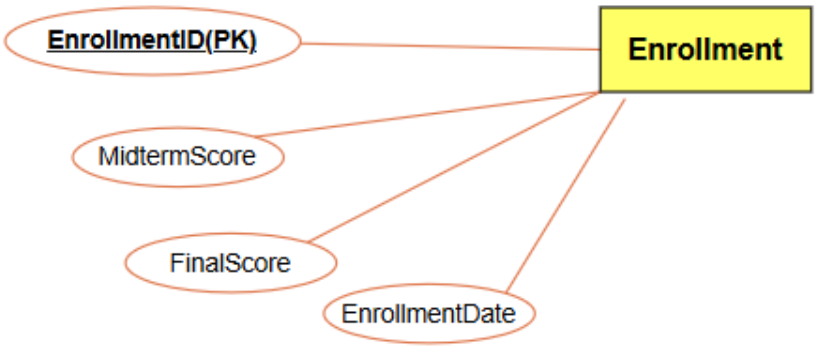
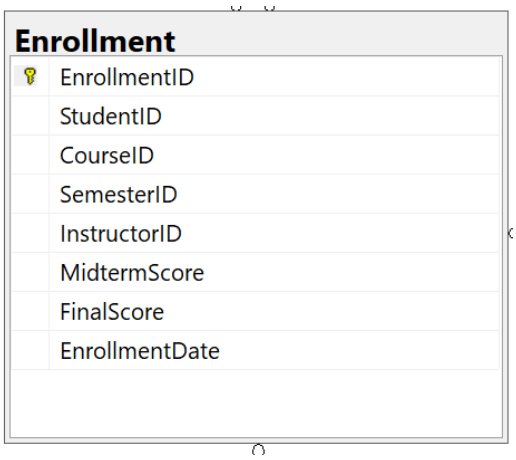



Figure 4.4:

This is the Enrollment entity, a junction table representing the many-to-many relationship between Student and Course. Enrollment has 8 attributes. EnrollmentID is the auto-incremented primary key. StudentID (FK) references the Student entity. CourseID (FK) references the Course entity. SemesterID (FK) identifies which semester the enrollment occurs. InstructorID (FK, nullable) identifies the instructor teaching this course section. MidtermScore and FinalScore store exam results (0-10 range), with NULL values allowed for pending scores. EnrollmentDate records the registration date. A UNIQUE constraint ensures one student cannot enroll in the same course twice per semester. This entity connects students to their coursework and enables grade tracking.




## 4.5 LEARNINGGOAL Entity

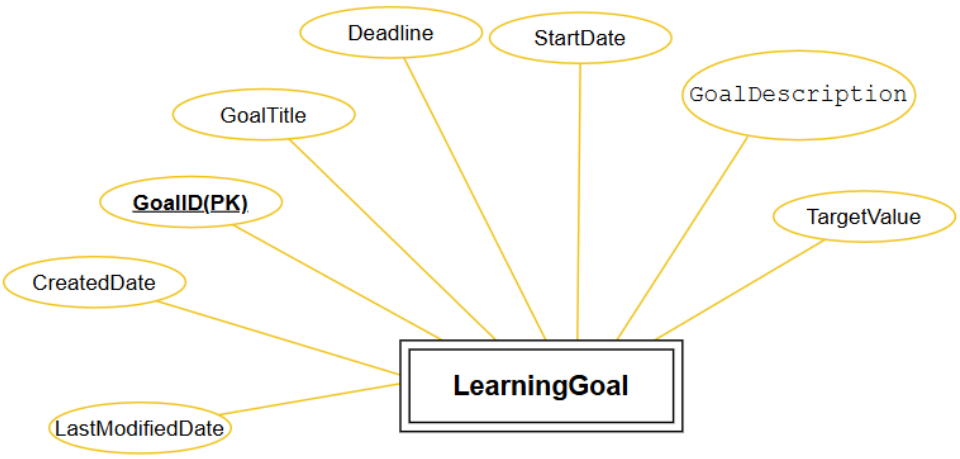
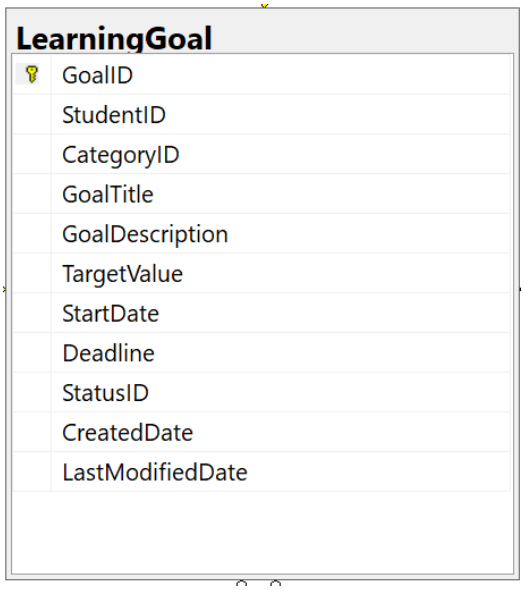



Figure 4.5:

This is the LearningGoal entity, representing student-defined objectives within the goal management system. LearningGoal has 11 attributes. GoalID is the auto-incremented primary key. StudentID (FK) references the student who owns the goal. CategoryID (FK) links to GoalCategoryLookup (Academic, Skill, Career, Personal, Project, Certificate). GoalTitle and GoalDescription define the goal. TargetValue specifies the measurable target (e.g., "GPA: 3.8", "Certificate Completion"). StartDate (default: TODAY) and Deadline define the goal timeframe. StatusID (FK) references GoalStatusLookup (Active, Completed, Failed, Cancelled). CreatedDate and LastModifiedDate track lifecycle. Each learning goal can have multiple progress tracking records and milestones, enabling detailed progress monitoring and goal decomposition.




## 4.6 PROGRESSTRACKING Entity

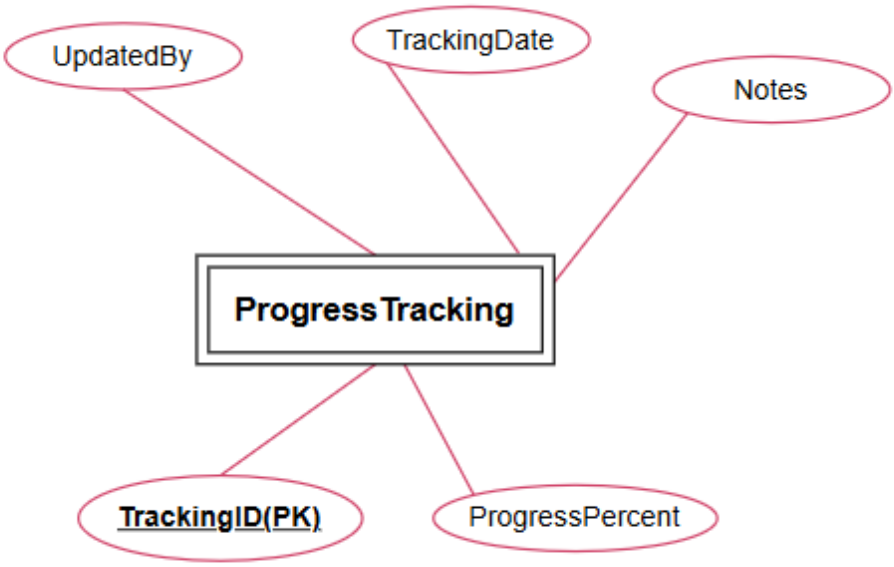
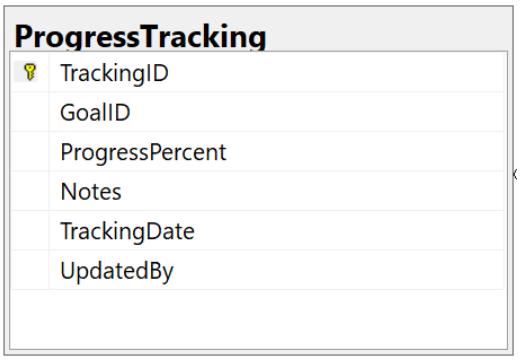



Figure 4.6:

This is the ProgressTracking entity, which maintains a historical audit trail of all progress updates for learning goals. ProgressTracking has 6 attributes. TrackingID is the auto-incremented primary key. GoalID (FK) references the parent LearningGoal entity. ProgressPercent stores the progress percentage (0-100) with decimal precision. Notes field allows textual descriptions of progress (e.g., "Completed module 1 on Udemy"). TrackingDate records when the update occurred (default: NOW). UpdatedBy is a VARCHAR field storing either StudentID or InstructorID (no FK to support both types). A CHECK constraint ensures ProgressPercent remains between 0 and 100. This entity uses ON DELETE CASCADE, meaning when a goal is deleted, all tracking records are automatically removed. Students and instructors can update progress collaboratively.




## 4.7 MILESTONE Entity

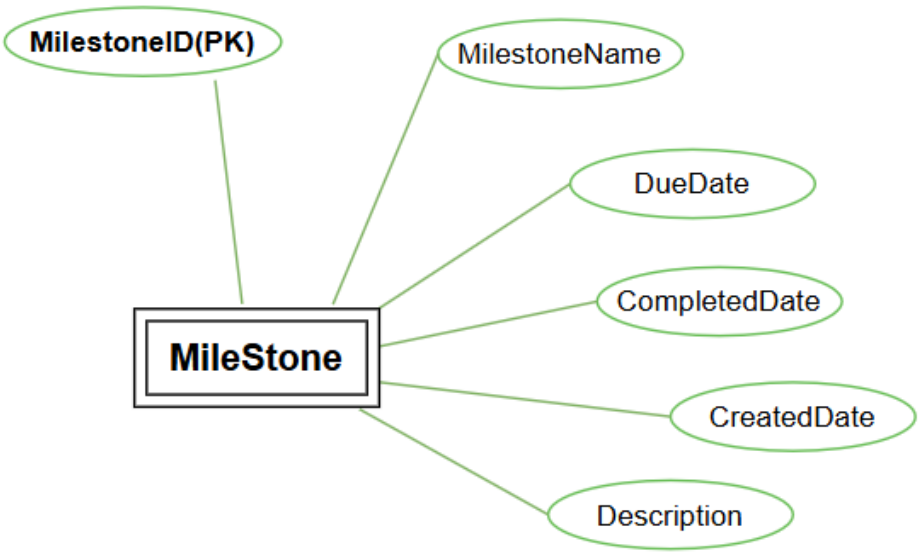
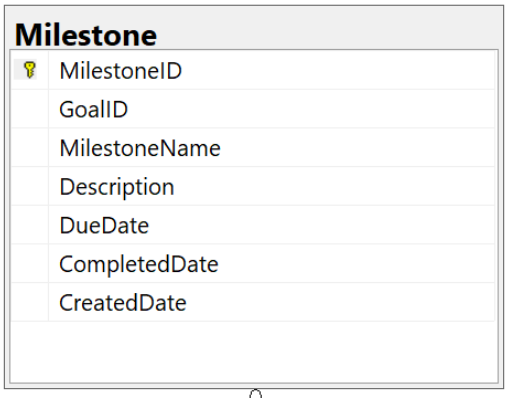



Figure 4.7:

This is the Milestone entity, representing sub-goals or checkpoints within a learning goal (weak entity). Milestone has 7 attributes. MilestoneID is the auto-incremented primary key. GoalID (FK) references the parent LearningGoal and is required (weak entity identity). MilestoneName provides the milestone name. Description explains the milestone. DueDate sets the target completion date. CompletedDate records actual completion (NULL if incomplete). CreatedDate records creation timestamp and is NOT NULL to support historical data insertion. A CHECK constraint ensures CompletedDate is NULL or $\geq$ CreatedDate. This entity uses ON DELETE CASCADE, so deleting a goal removes all milestones. Milestones enable goal decomposition into manageable sub-tasks with individual tracking.




## 4.8 ADVISORASSIGNMENT Entity

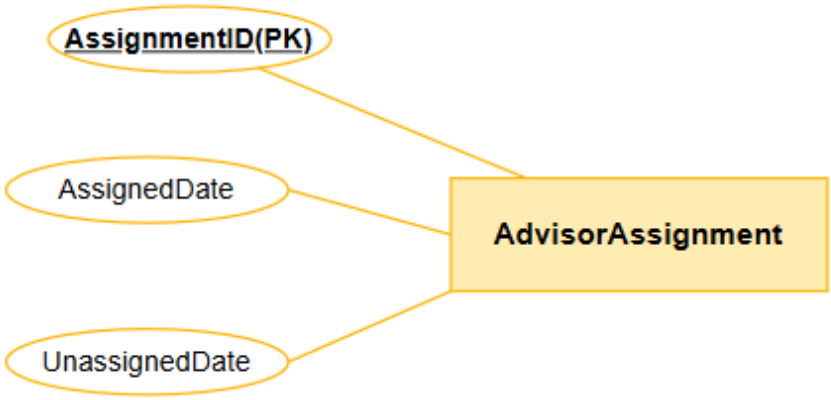
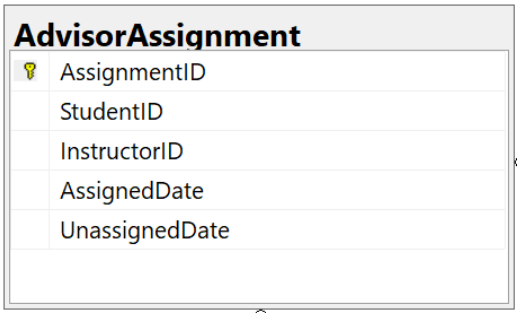



Figure 4.8:

This is the AdvisorAssignment entity, managing the many-to-many relationship between students and instructors with temporal validity. AdvisorAssignment has 5 attributes. AssignmentID is the auto-incremented primary key. StudentID (FK) references the student being advised. InstructorID (FK) references the assigned advisor (instructor). AssignedDate records when the advising relationship began (default: TODAY). UnassignedDate records when the relationship ended (NULL if currently active). A CHECK constraint ensures UnassignedDate is NULL or > AssignedDate. IsActive is derived: UnassignedDate IS NULL = Active. This temporal design enables auditing of historical advisor relationships. Both FKs use ON DELETE NO ACTION to prevent accidental deletion of student/instructor records. The entity supports multiple advisors per student and multiple students per advisor across time.




## 4.9 LOOKUP TABLES

Lookup tables contain reference data that doesn't change frequently and provides controlled vocabularies for the system.

### 4.9.1 GOALCATEGORYLOOKUP

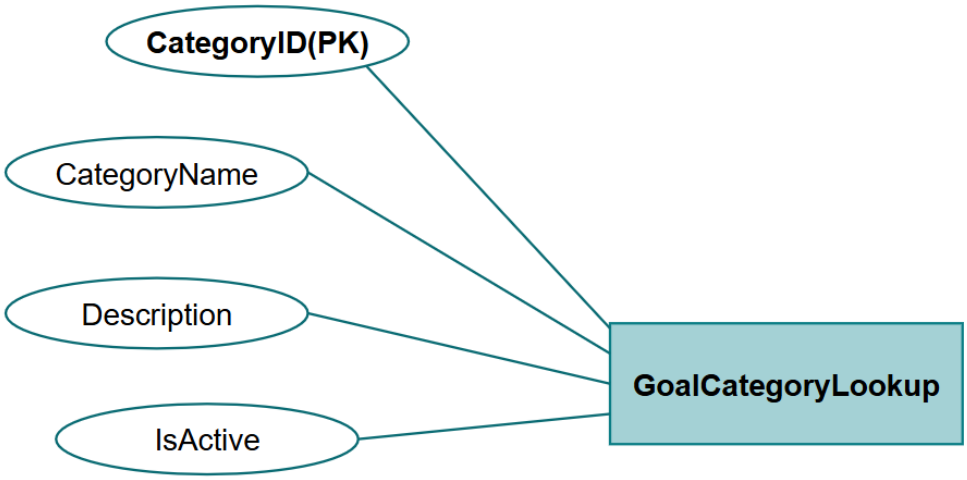
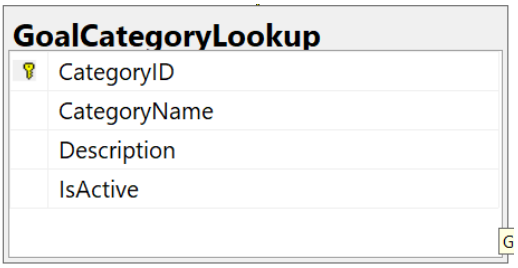



Figure 4.9:

GoalCategoryLookup is a master lookup table containing predefined goal categories for classification. It has 4 attributes: CategoryID (PK, IDENTITY), CategoryName (UNIQUE), Description, and IsActive. Standard values include: Academic (study/GPA goals), Skill Development (professional skills), Career (internships/employment), Personal (health/improvement), Project (assignments), and Certificate (professional certifications). This lookup enables consistent goal categorization across the system and prevents data anomalies from invalid category values. The UNIQUE constraint on CategoryName ensures no duplicate categories exist.




### 4.9.2 GOALSTATUSLOOKUP

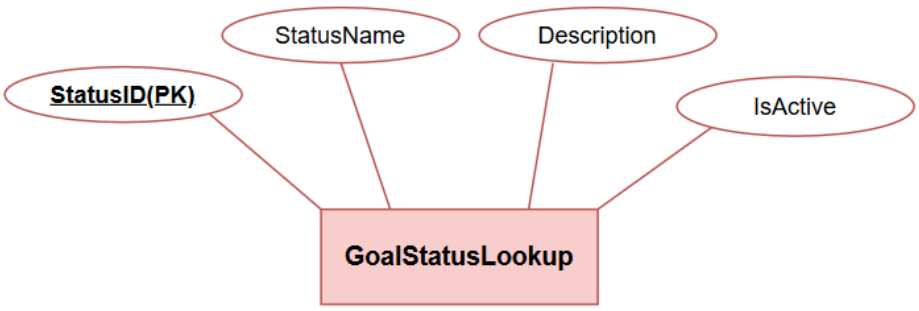
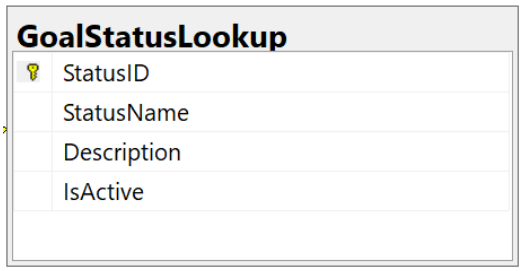



Figure 4.10:

GoalStatusLookup is a master lookup table containing goal state values. It has 4 attributes: StatusID (PK, IDENTITY), StatusName (UNIQUE, CHECK constraint), Description, and IsActive. Allowed status values are: Active (goal in progress), Completed (successfully achieved), Failed (not achieved), and Cancelled (explicitly terminated). The CHECK constraint restricts StatusName to these four values, enforcing data integrity at the database level. This prevents invalid goal statuses and ensures consistent state tracking throughout the system.



### 4.9.3 SEMESTERLOOKUP

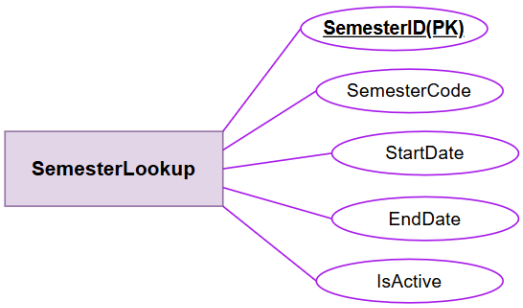
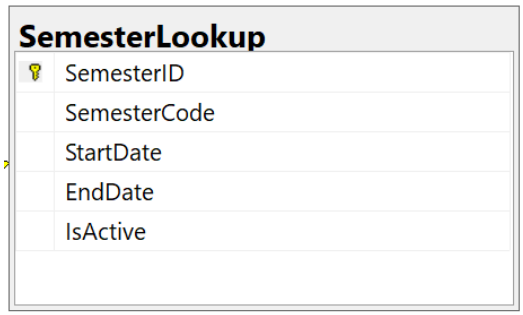



Figure 4.11:

SemesterLookup is a master lookup table containing academic semester reference data. It has 5 attributes: SemesterID (PK, IDENTITY), SemesterCode (UNIQUE), StartDate, EndDate, and IsActive. SemesterCode uses standardized format (e.g., "Fall2024", "Spring2025"). A CHECK constraint ensures EndDate > StartDate, preventing invalid date ranges. Example data includes: Fall2024 (09/01–12/31), Spring2025 (01/01–05/31), Summer2025 (06/01–08/31), Fall2025 (09/01–12/31). This lookup enables temporal organization of enrollments, goals, and progress tracking across academic periods.




## 4.10 NORMAL FORM

BEFORE (Violates 3NF): COURSE: CourseID CourseName, Credits, InstructorID, InstructorName
Transitive: CourseID InstructorID InstructorName

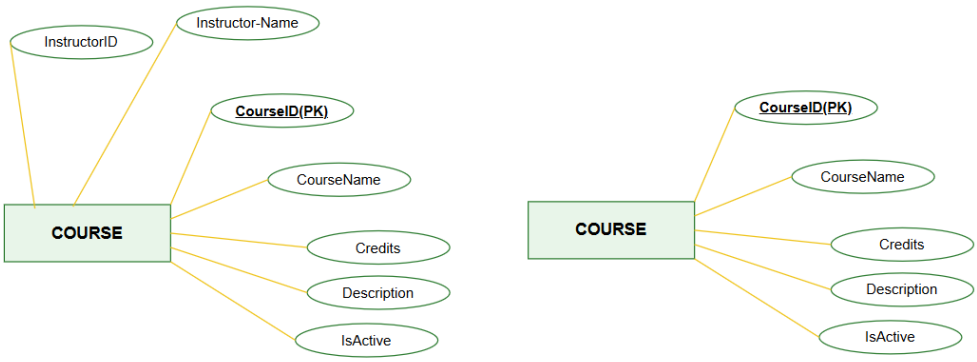



Figure 4.12:

AFTER (3NF): COURSE: CourseID  CourseName, Credits ENROLLMENT: StudentID, CourseID, SemesterID  InstructorID No transitive dependencies

## 4.11 FULL DIAGRAM

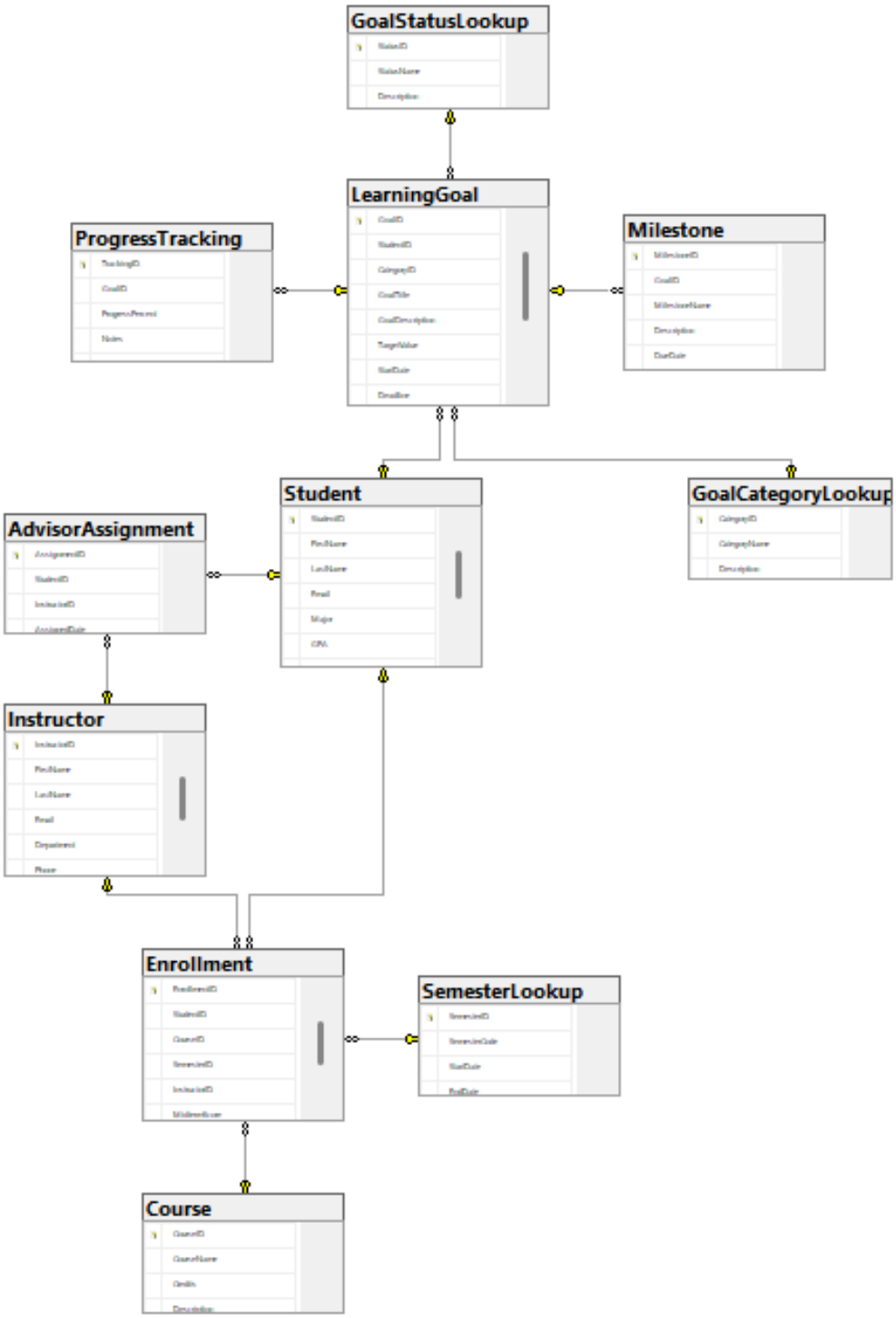



Figure 4.13: Full Diagram

# Chapter 5

# SQL COMMAND

## 5.1 Query Using ORDER BY

**Topic:** Sorting students by academic performance

Listing 5.1: ORDER BY Example

```sql
SELECT
    StudentID,
    FirstName + ' ' + LastName AS FullName,
    Major,
    GPA
FROM Student
ORDER BY GPA DESC, StudentID ASC;
```

**Purpose of the query:** The Academic Affairs Office needs a comprehensive list of all students in the school, prioritizing those with the highest academic achievements. This list will be used to prepare for scholarship distribution and recognition ceremonies. Students must be sorted in descending order by GPA (highest to lowest), with StudentID as the secondary sort key in ascending order.
**Result:**

Results | Messages

| | StudentID | FullName | Major | GPA |
|---|---|---|---|---|
| 1 | HE160003 | Bui Linh H | Information Systems | 3.95 |
| 2 | SS160001 | Vu Thanh F | Software Engineering | 3.80 |
| 3 | SE160002 | Tran Thi B | Software Engineering | 3.75 |
| 4 | HE160001 | Pham Thi D | Information Systems | 3.60 |
| 5 | SE160001 | Nguyen Van A | Software Engineering | 3.50 |
| 6 | HE160002 | Hoang Minh E | Information Systems | 3.40 |
| 7 | SE160003 | Le Van C | Software Engineering | 3.20 |
| 8 | SE160004 | Dang Hong G | Software Engineering | 2.90 |

Figure 5.1:

## 5.2 Query Using INNER JOIN

**Topic:** Retrieving detailed student transcript

Listing 5.2: INNER JOIN Example

```sql
SELECT
    s.FirstName,
    s.LastName,
    c.CourseName,
    e.MidtermScore,
    e.FinalScore
FROM
    Student s
INNER JOIN
    Enrollment e ON s.StudentID = e.StudentID
INNER JOIN
    Course c ON e.CourseID = c.CourseID
WHERE
    s.StudentID = 'SE160002';
```

**Purpose of the query:** Student 'SE160002' wants to view his/her complete transcript. This query joins three tables (Student, Enrollment, and Course) to display the student's first name, last name, course names, midterm scores, and final scores for all courses the student has enrolled in.
**Result:**

Results | Messages

| | FirstName | LastName | CourseName | MidtermScore | FinalScore |
|---|---|---|---|---|---|
| 1 | Tran | Thi B | Introduction to Databases | 9.00 | 9.50 |
| 2 | Tran | Thi B | Software Engineering | 8.00 | 8.50 |
| 3 | Tran | Thi B | Machine Learning Engineering | 8.80 | 9.20 |

Figure 5.2:

## 5.3 Query Using AGGREGATE FUNCTIONS

**Topic:** School KPI (Key Performance Indicators) statistics

Listing 5.3: AGGREGATE FUNCTIONS Example

```sql
SELECT
    (SELECT COUNT(*)
     FROM Student
     WHERE IsActive = 1) AS TotalActiveStudents,
    (SELECT AVG(CAST(GPA AS DECIMAL(5,2))) FROM Student) AS AverageGPA,
    (SELECT COUNT(*)
     FROM LearningGoal lg
     JOIN GoalStatusLookup gs ON gs.StatusID = lg.StatusID
     WHERE gs.StatusName = 'Active') AS TotalActiveGoals;
```

**Purpose of the query:** The school dashboard needs to display three key performance indicators (KPI):

- *TotalActiveStudents:* Count of all active students (IsActive = 1)
- *AverageGPA:* Average GPA of all students
- *TotalActiveGoals:* Total count of learning goals currently in 'Active' status

This single query returns all three metrics using correlated subqueries with aggregate functions. **Result:**

Results | Messages

| | ToTalActiveStudents | AverageGPA | TotalActiveGoals |
|---|---|---|---|
| 1 | 8 | 3.512500 | 11 |

Figure 5.3:

## 5.4 Query Using GROUP BY and HAVING Clauses

**Topic:** Instructor performance evaluation

Listing 5.4: GROUP BY and HAVING Example

```sql
SELECT
    e.InstructorID,
    AVG(e.FinalScore) AS AvgFinalScore,
    COUNT(*) AS NumEnrollments
FROM Enrollment e
GROUP BY e.InstructorID
HAVING AVG(e.FinalScore) > 8.5
    AND COUNT(*) >= 2
ORDER BY AvgFinalScore DESC;
```

**Purpose of the query:** The department wants to identify instructors with strong teaching performance based on student grades. This query groups enrollment records by instructor and filters to show only instructors whose average final score exceeds 8.5 AND have taught at least 2 courses. Results are sorted by average score in descending order to highlight top performers. **Result:**

Results | Messages

| | InstructorID | AvgFinalScore | NumEnrollments |
|---|---|---|---|
| 1 | INS00001 | 8.750000 | 6 |

Figure 5.4:

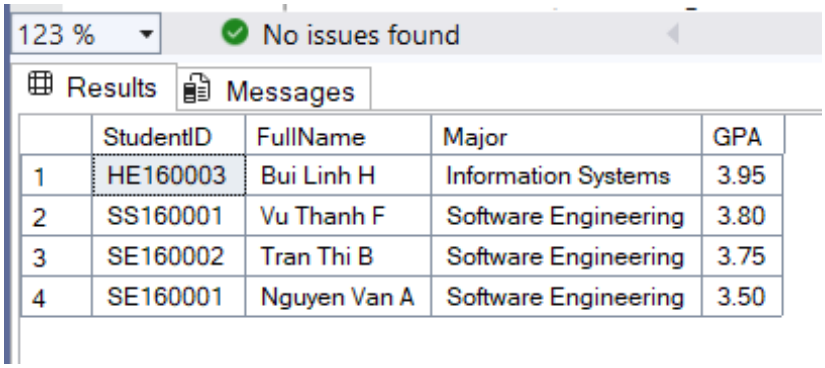
## 5.5 Query Using a Sub-query as a Relation

**Topic:** Finding outstanding students (compared to major average)

Listing 5.5: Sub-query as Relation Example

```sql
SELECT
    s.StudentID,
    s.FirstName + ' ' + s.LastName AS FullName,
    s.Major,
    s.GPA
FROM Student s
INNER JOIN (
    SELECT Major, AVG(GPA) AS AvgMajorGPA
    FROM Student
    GROUP BY Major
) mg ON s.Major = mg.Major
WHERE s.GPA > mg.AvgMajorGPA
ORDER BY s.Major, s.GPA DESC;
```

**Purpose of the query:** Identify high-achieving students whose GPA is higher than the average GPA of their major program. This query uses a derived table (subquery in FROM clause) to calculate the average GPA for each major, then joins it with the Student table to compare individual students against their major's average. Only students exceeding their major's average GPA are displayed.
**Result:**




| | StudentID | FullName | Major | GPA |
|---|---|---|---|---|
| 1 | HE160003 | Bui Linh H | Information Systems | 3.95 |
| 2 | SS160001 | Vu Thanh F | Software Engineering | 3.80 |
| 3 | SE160002 | Tran Thi B | Software Engineering | 3.75 |
| 4 | SE160001 | Nguyen Van A | Software Engineering | 3.50 |

Figure 5.5:

## 5.6 Query Using Partial Matching in WHERE Clause

**Topic:** Building a learning goal search feature

Listing 5.6: Partial Matching Example

```sql
SELECT
    GoalID,
    GoalTitle
FROM
    LearningGoal
WHERE
    (GoalTitle LIKE N'%Docker%' OR GoalTitle LIKE N'%Kubernetes%')
```

```
    OR
    (GoalDescription LIKE N'%Docker%' OR GoalDescription LIKE N'%
       Kubernetes%');
```

**Purpose of the query:** Users (students or advisors) need to search for learning goals related to DevOps skill development. This query returns GoalID and GoalTitle of all goals where either the GoalTitle or GoalDescription contains the keywords 'Docker' OR 'Kubernetes'. The LIKE operator with wildcard (%) enables flexible partial string matching across multiple columns. **Result:**

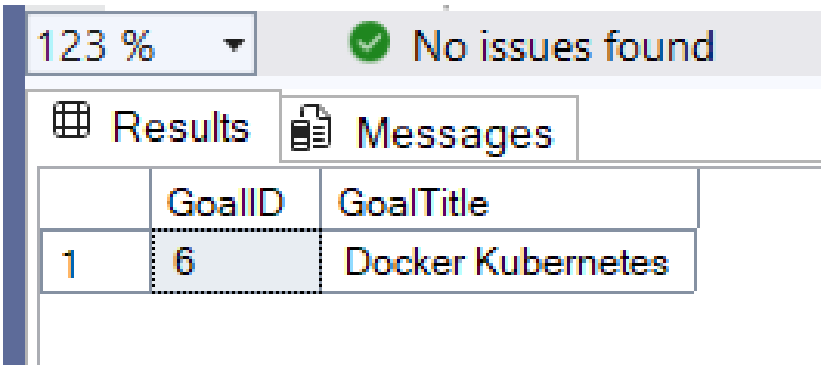



Figure 5.6:

## 5.7 Query Using a Self-Join

**Topic:** Finding pairs of students in the same class

Listing 5.7: Self-Join Example

```
SELECT
    e1.CourseID,
    e1.StudentID AS Student1_ID,
    e2.StudentID AS Student2_ID
FROM
    Enrollment e1
INNER JOIN
    Enrollment e2 ON e1.CourseID = e2.CourseID
    AND e1.SemesterID = e2.SemesterID
WHERE
    e1.StudentID < e2.StudentID;
```

**Purpose of the query:** Identify pairs of students who have enrolled in the same course (CourseID) during the same semester (SemesterID). This query uses a self-join on the Enrollment table with two aliases (e1 and e2). Critical considerations:

- The WHERE clause ensures a student is never paired with themselves (e1.StudentID < e2.StudentID)
- Each pair appears only once (avoids showing both (SE160001, SE160002) and (SE160002, SE160001))
- Results display CourseID, Student1_ID, and Student2_ID

**Result:**

123 % No issues found

Results Messages

| | CourseID | Student1_ID | Student2_ID |
|---|---|---|---|
| 1 | DBI202 | HE160002 | SE160001 |
| 2 | DBI202 | HE160002 | SE160002 |
| 3 | DBI202 | SE160001 | SE160002 |
| 4 | WEB301 | SE160001 | SS160001 |

Figure 5.7:

## 5.8 Stored Procedure

**Topic:** Creating a stored procedure to add a new course

Listing 5.8: Stored Procedure Example

```sql
CREATE PROCEDURE sp_AddNewCourse
    @CourseID VARCHAR(10),
    @CourseName NVARCHAR(200),
    @Credits INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        INSERT INTO Course (CourseID, CourseName, Credits)
        VALUES (@CourseID, @CourseName, @Credits);

        PRINT 'New course has been added successfully.';
        SELECT *
        FROM Course
    END TRY
    BEGIN CATCH
        PRINT 'Error: ' + ERROR_MESSAGE();
        RAISERROR('Unable to add course. An error has occurred.', 16, 1);
    END CATCH
END;
-- Example execution:
EXEC sp_AddNewCourse @CourseID = 'NEW101', @CourseName = N'New Course
   Title', @Credits = 3;
```

**Purpose of the query:** This procedure automates the process of adding a new course to the Course table. It accepts three input parameters (CourseID, CourseName, Credits) and inserts them into the database. Key points:

- Fields not provided (Description, IsActive) will use their table defaults
- Includes basic error handling with TRY-CATCH block
- Uses SET NOCOUNT ON to suppress system messages
- Provides success and error messages
- Show all information of Course in database

**Result:**

Results | Messages

| | CourseID | CourseName | Credits | Description | IsActive |
|---|---|---|---|---|---|
| 1 | AID101 | Artificial Intelligence Basics | 3 | Introduction to AI and machine learning | 1 |
| 2 | DBI202 | Introduction to Databases | 3 | Database design, SQL, normalization, Chapter 3-6 | 1 |
| 3 | DSA201 | Data Structures and Algorithms | 3 | Data structures and algorithm optimization | 1 |
| 4 | MAE101 | Mathematics for Engineers | 4 | Advanced mathematics, linear algebra, calculus | 1 |
| 5 | MLE401 | Machine Learning Engineering | 3 | Neural networks, deep learning, TensorFlow | 1 |
| 6 | NEW101 | New Course Title | 3 | NULL | 1 |
| 7 | PRJ301 | Java Web Application | 3 | Web application development with Java and Spring | 1 |
| 8 | SWE201 | Software Engineering | 3 | Software development process, Agile, and SDLC | 1 |
| 9 | WEB301 | Web Development Advanced | 3 | Advanced web technologies React, Angular, Node.js | 1 |

Figure 5.8:

## 5.9 Trigger (Audit Logging)

**Topic:** Creating a trigger for student insertion audit logging

Listing 5.9: Trigger Example

```sql
CREATE TRIGGER trg_AuditNewStudent
ON Student
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;


    INSERT INTO AuditLog (
        TableName,
        Operation,
        RecordID,
        NewValue,
        ModifiedBy
    )


    SELECT
        'Student',
        'INSERT',
        i.StudentID,
        i.FirstName + ' ' + i.LastName,
        'SYSTEM'
    FROM
        inserted i;
END
```

**Purpose of the query:** This trigger automatically creates an audit log entry whenever a new student is added to the system. It captures:

- *TableName:* 'Student'

- *Operation:* 'INSERT'
- *RecordID:* The new StudentID
- *NewValue:* Student's full name (FirstName + LastName concatenation)
- *ModifiedBy:* 'SYSTEM' (indicating automatic enrollment action)

The trigger uses the special 'inserted' table to access values of newly added records, ensuring audit compliance and data tracking for system transparency.

# THE END